ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) | 979-8-3503-6874-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICASSP49660.2025.10889328

MixHD: A Method for Detecting Hallucinations Based on the Internal State and Output Probability of Large Language Models

Chuang Li[1,2], Bingnan Xing[3], Dongdong Huo[1,2†], Qihui Zhou[1,2], Zhen Xu[1,2] and Yu Wang[1,2]
[1]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3]School of Mathematics and Statistics, Henan University, Zhengzhou, China

***Abstract*—This paper presents a novel hallucination detection method based on the internal states and output probabilities of large language models (LLMs) to address the common issue of hallucinations in model-generated content. We designed a new detection framework that extracts internal features such as hidden layer states and word probabilities, and analyzes the output probability distribution to more accurately identify potential hallucinations. Experimental results show that our method outperforms traditional hallucination detection approaches across multiple evaluation metrics and performs well on our EpicQA dataset. This method offers a new perspective for hallucination detection and makes a significant contribution to improving the reliability of language models.**

***Index Terms*—LLMs, hallucination detection, probability distributions, internal states**

I. Introduction

Large language models (LLMs) have achieved remarkable success in various natural language processing tasks, including text generation [1], translation, and question answering. However, despite their powerful capabilities, LLMs often produce outputs that are factually inaccurate, logically inconsistent, or misleading, a phenomenon referred to as "hallucination." [2] This issue poses significant challenges in high-risk applications such as healthcare [3], legal consultation [4], and automated decision-making, where incorrect information can lead to severe consequences.

Current methods for detecting hallucinations primarily rely on post-processing techniques, such as consistency checks, external validation against knowledge bases [5], or heuristic-based filtering. However, these approaches often fall short due to their reliance on external resources, lack of generalizability, and inability to fully leverage the internal mechanisms of LLMs [6]. As LLMs become increasingly complex, it is critical to address hallucinations by directly understanding and tackling them through the model's internal processes.

At present, hallucination detection methods in the absence of external knowledge sources are mainly divided into two categories: internal state-based approaches and behavior-based approaches [7]. Internal state-based methods typically detect hallucinations by analyzing token probabilities [8] or entropy [9], while behavior-based methods sample multiple responses to the same prompt and evaluate the consistency between them. However, these methods often evaluate LLMs from a single perspective, failing to capture the full spectrum of information processing from input to output.

In this paper, we propose a novel hallucination detection method that leverages both the internal states and output probabilities of LLMs. By examining hidden layer states, attention mechanisms, and output probability distributions, our approach aims to more effectively utilize the model's inherent features to identify hallucinations. This method not only improves detection accuracy but also offers deeper insights into the root causes of hallucinations, paving the way for more robust and reliable language models. We aim to advance the field of hallucination detection in LLMs, ultimately enhancing the reliability and safety of LLM applications. Our main contributions are:

- Introduced a new framework that combines internal state analysis with output probability modeling for hallucination detection.
- We conduct extensive experiments to validate the effectiveness of our method, demonstrating its superior performance over existing approaches.
- We created a question-answer dataset (EpicQA) based on the epic literary work *Mahabharata* and tested our method on this dataset. The results demonstrate that our method achieved strong performance.

II. Related Work

Current research on hallucination detection for large language models (LLMs) in zero-knowledge scenarios can be broadly categorized into methods based on internal states and those based on model behavior [10] [11]. Internal state methods mainly involve analyzing hidden layer activation vectors and word probabilities in model outputs. Azaria et al. [6] proposed a technique for detecting the factuality of LLM statements by examining the model's internal states. They trained a classifier that predicts the probability of a

† Dongdong Huo is the corresponding author. huodongdong@iie.ac.cn

Authorized licensed use limited to: M S RAMAIAH INSTITUTE OF TECHNOLOGY. Downloaded on October 01,2026 at 09:59:34 UTC from IEEE Xplore. Restrictions apply.

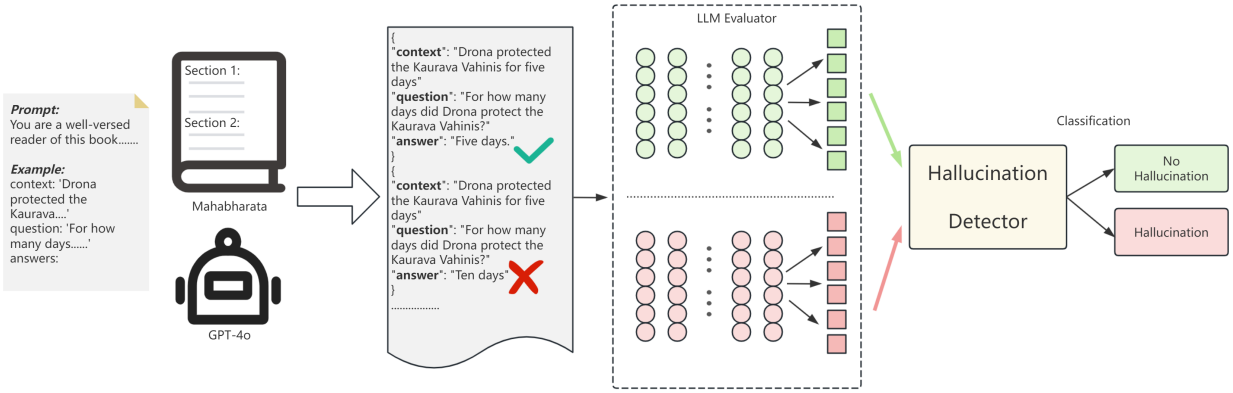


Fig. 1. Overall Detection Process

statement being true by analyzing the hidden layer activations when the LLM generates a statement. Similarly, Su et al. [12] introduced an unsupervised, real-time hallucination detection method based on the internal states of large language models. Their work revealed that comparing the impact of context embeddings from different layers on the output shows that the probability of the last token in the final layer's context embedding is effective in identifying hallucinations [13].

Probabilities from LLMs have been effective in detecting hallucinations. Quevedo et al. [8] used word probabilities and four numerical features from LLM evaluators to train classifiers that identify hallucinations. Farquhar et al. [7] introduced an entropy-based method that clusters multiple outputs from the same query and calculates semantic entropy to detect hallucinations. However, this and the SelfCheckGPT method by Manakul et al. [14] require multiple responses, reducing efficiency.

In light of the current research on hallucination detection for large language models in zero-knowledge scenarios, this paper proposes the MixHD method. By integrating internal embedding states with output word probabilities of LLMs, the MixHD method effectively leverages the intrinsic features of LLMs to enhance hallucination detection, providing improved performance in zero-knowledge scenarios.

III. METHODOLOGY

We propose a hallucination detection method based on the internal states and output probabilities of large language models (LLMs), aiming to leverage the intrinsic characteristics of LLMs, such as hidden layer activations, attention mechanisms, and output distributions, to detect inconsistencies or inaccuracies in generated content without relying on external knowledge sources. As shown in Figure 1, by extracting vectors from the hidden layers, integrating attention mechanisms, and analyzing the entropy distribution of the outputs, we train a binary classification model that effectively performs hallucination detection.

A. Problem Formulation

Hallucination detection is essentially a binary classification problem, where the detection of a hallucination by the detector is labeled as 1, and the absence of a hallucination is labeled as 0. Current hallucination detection primarily involves large language models in tasks such as knowledge-based question answering, text summarization, dialogue question answering, and text generation. By selecting common LLM evaluators and testing on well-constructed datasets, features are extracted from within the large models. These features are then used to retrain a detector, effectively leveraging the model's intrinsic characteristics for hallucination detection.

B. Feature Selection

The first step of this method is to extract relevant internal states of the LLM during the generation process, mainly including the hidden layer activations of different layers and the activation of the last token in the final layer. These internal features are considered to contain critical information related to the model's decision-making process. According to Su et al. [12], their experiments found that by extracting the contextual embedding vector of the last token in the final layer, hallucinations could be effectively distinguished. Therefore, we focus our analysis solely on the hidden states of the final layer and the contextual embedding vector of the last token in the final layer.

1) The All Contextualized Embedding (ACE): To extract activation states from multiple hidden layers of the model, we compute the average of the contextualized embedding between the hidden states of different layers, which serves as the hidden layer feature of the model. Let the hidden state activation of the model be H_i, where i represents the i-th layer, and there are N hidden layers in total. The formula for computing the average contextualized embedding of these hidden states can be expressed as:

$$H_{\text{avg}} = \frac{1}{N} \sum_{j=1}^{N} \frac{1}{n} \sum_{i=1}^{n} H_j^i \tag{1}$$

where H_{avg} represents the average contextualized embedding of all hidden layer activations, H_j^i is the i-th token in the hidden state activation of the j-th layer, and N is the total number of hidden layers.

Authorized licensed use limited to: M S RAMAIAH INSTITUTE OF TECHNOLOGY. Downloaded on October 01,2026 at 09:59:34 UTC from IEEE Xplore. Restrictions apply.

2) The Final Token's Embedding (FTE): The last token in the final hidden layer is the result of the model's processing of the last token in the input sequence at the last layer, and it plays a significant role in the generation of the final output. Here, we define the embedding vector of the last token in the final layer as h_N^n and take its average across all layers.

$$h_{avg} = \frac{1}{N}\sum_{j=1}^{N} h_j^n \tag{2}$$

C. Output Probability

In addition to analyzing the hidden layer activation vectors, we also utilize the model's output probability distribution to assess its confidence in the generated content. The output probabilities reflect the model's confidence in each generated word, while entropy indicates the level of dispersion in the data. A higher entropy value suggests greater dispersion and higher uncertainty, whereas a lower entropy value indicates more concentration and higher certainty. We hypothesize that hallucinated outputs are often accompanied by lower confidence, which manifests as uncertainty or a flat distribution in the probability distribution.

1) Token Probability (TP): In the generation task, the model outputs unnormalized scores called "**logits**" in the final layer. We use the softmax function to convert the logits into probabilities, ensuring that the sum of the probabilities for all words equals 1. The calculation is as follows:

$$P(w_i) = \frac{exp(logit_i)}{\sum_{j=1}^{V} exp(logit_j)} \tag{3}$$

where $P(w_i)$ represents the probability of the word w_i, and $logit_i$ is the logit value for the world w_i, with V being the vocabulary size. We use the greedy search algorithm to select the word with the highest probability from the softmax probability distribution.

2) Entropy (En): Entropy represents the degree of dispersion of a probability distribution. A higher entropy value indicates that the data is more concentrated, resulting in a higher confidence in the output. Conversely, when the data is more dispersed, the confidence in the output is lower. The formula for calculating entropy is as follows:

$$H(p) = -\sum_{i=1}^{V} P(w_i)\log P(w_i) \tag{4}$$

where $P(w_i)$ is the probability of the token w_i, V represents the vocabulary size. By monitoring entropy values, we can understand the generation quality and uncertainty of the model.

D. Model

We combined the extracted features and trained a Multilayer Perceptron (MLP) network for hallucination detection. In terms of model configuration, the input size is set to 4 (ACE, FTE, TP, En), corresponding to the four features mentioned earlier. The model consists of three hidden layers, with sizes of 256, 128, and 64, respectively. The Rectified Linear Unit (ReLU) is used as the activation function between the hidden layers. The output layer is activated using the sigmoid function to better suit the binary classification task. For the loss function, we employed the cross-entropy loss function, which is expressed as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1-y_i)\log(1-p_i)] \tag{5}$$

where y_i represents the actual label of the i-th example, p_i is the probability that the model predicts the i-th sample as 1.

E. The EpicQA Dataset

In this section, we will introduce the question-answer dataset we created based on the epic work MAHABHARATA. This dataset contains 1,000 question-answer pairs, with each pair consisting of (context, question, correct answer, incorrect answer). Due to the extensive content of the text, we selected only a portion of it to create the dataset.Our dataset can be found here: https://github.com/jeremery/EpicQA

During the dataset generation process, we used the GPT-4o model to generate the question-answer pairs. First, the document was divided into chapters, and we created 30 sections in total. Then, we constructed appropriate prompts to have GPT-4o generate several questions and answers for each chapter. The prompt template we used is as follows:

a) prompt: You are a well-versed reader of this book and an expert in the field of question-answer dataset generation. Based on the following content, please generate 10 questions suitable for a question-answer dataset. The context and the question should be as concise as possible, and both the context and question must include the key information from the provided content. For these 10 questions, generate the corresponding question-answer pairs, including the context, question, right answer, and wrong answer.

b) example: {context: "Drona protected the Kaurava Vahinis for five days.", question: "For how many days did Drona protect the Kaurava Vahinis?", right_answer: "Five days.", wrong_answer: "Ten days." }

IV. EXPERIMENTAL SETTINGS

A. Dataset and Metrics

We primarily used the HELM dataset [12] and the EpicQA dataset for validation. The main metrics selected were AUC (Area Under the Curve) and Accuracy. The HELM dataset includes models such as Falcon-40B [15], GPT-J-6B [16], LLaMA-2-Base-7B [17], LLaMA-2-Chat-7B [18], and OPT-6.7B [19]. Each model is accompanied by a data.json file and corresponding hidden layer states. Therefore, we chose HELM as our evaluation dataset.

B. Baseline

To validate our approach, we conducted extensive experiments on four LLM architectures: OPT-7, OPT-13, LLaMA-2, and Falcon-40. We evaluated the performance of these models on the two datasets mentioned above to verify our method. The traditional methods we selected for comparison include:

Authorized licensed use limited to: M S RAMAIAH INSTITUTE OF TECHNOLOGY. Downloaded on October 01,2026 at 09:59:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE RESULTS OF DIFFERENT MODELS ON DIFFERENT DATASETS AND METHODS

	HELM AUC				HELM ACC			
Baselines	**OPT-7B**	**LLC-7B**	**GPT-J**	**Falcon-40B**	**OPT-7B**	**LLC-7B**	**GPT-J**	**Falcon-40B**
PP	0.7302	0.4166	0.7074	0.5749	0.6962	0.4902	0.7533	0.7232
PE	0.7263	0.4931	0.7497	0.6479	0.7506	0.6922	0.7538	0.7621
SAPLMA	0.6212	0.4066	0.6987	0.5128	0.8031	0.5430	0.7912	0.7142
MIND	0.8835	0.6755	0.8774	**0.7895**	0.7287	0.7322	0.8029	0.8571
MixHD(Ours)	**0.9113**	**0.7121**	**0.8901**	0.7612	**0.8823**	**0.8581**	**0.8876**	**0.8623**
	EpicQA AUC				**EpicQA ACC**			
Baselines	**OPT-7B**	**LLC-7B**	**GPT-J**	**Falcon-40B**	**OPT-7B**	**LLC-7B**	**GPT-J**	**Falcon-40B**
PP	0.7821	0.5482	0.7432	0.6321	0.7192	0.5321	0.7632	0.7532
PE	0.7681	0.5090	0.7312	0.6574	0.7302	0.5791	0.7527	0.7302
SAPLMA	0.6918	0.5332	0.7098	0.5877	0.7871	0.6212	0.7873	0.7286
MIND	0.7321	0.6421	0.7991	**0.7382**	0.7561	0.7321	0.8101	0.8129
MixHD(Ours)	**0.8012**	**0.7312**	**0.8313**	0.7102	**0.8231**	**0.8511**	**0.8821**	**0.8765**

TABLE II
PERFORM ABLATION ANALYSIS ON DIFFERENT FEATURES ON THE HELM

Features				AUC			
ACE	**FTE**	**TP**	**En**	**OPT-7B**	**LLC-7B**	**GPT-J**	**Falcon-40B**
✓	✓	✓	✓	**0.9113**	**0.7121**	**0.8901**	**0.7612**
✓				0.8873	0.6921	0.8521	0.7012
	✓			0.8901	0.6871	0.8671	0.7121
		✓		0.8023	0.5021	0.7321	0.6023
			✓	0.7562	0.5381	0.7561	0.6321

1) Predictive Probability (PP): introduced by Manakul et al. [20], is a method used to estimate the confidence of a model's predictions by examining the likelihood of predicted tokens or outputs. In essence, PP measures the probability that a model assigns to its predicted answer or sequence, providing an indication of how confident the model is in its prediction.

2) Predictive Entropy (PE): introduced by Kadavath et al. [9], measures the uncertainty of a model's predictions by calculating the entropy of the predicted probability distribution. A higher entropy indicates greater uncertainty in the model's output, while lower entropy suggests more confident and concentrated predictions.

3) SAPLMA [6]: is a method designed to detect hallucinations in large language models (LLMs). It works by training a classifier using the activation values from the hidden layers of LLMs. By leveraging these internal representations, SAPLMA aims to identify when the model generates outputs that are not grounded in the input or context, effectively flagging hallucinated content.

4) MIND [12]: is a method that decodes neural activity by maximizing the mutual information between the input and the neural responses, enabling more accurate interpretation of brain signals or model behavior by preserving the most relevant information in the decoding process.

V. RESULTS AND ANALYSIS

The experimental results (shown in Table I) demonstrate that our method significantly improves hallucination detection compared to baseline approaches. The performance of the Falcon-40B model on MIND is slightly higher than our method, which may be attributed to differences in model architecture. By directly utilizing internal states and output probabilities, our method offers a more integrated and efficient hallucination detection solution, reducing reliance on external resources and enhancing the reliability of LLM outputs in zero-knowledge scenarios.

In Table II, we individually analyzed four features (ACE, FTE, TP, En) to assess their impact on the final results. The experiments revealed that ACE, i.e., the embedding vectors of all hidden layers in the model, had the least impact on the outcome, followed by FTE, indicating that ACE is of higher importance. The features with the greatest impact on the results were TP and En, which were less critical in comparison.

VI. CONCLUSIONS AND FUTURE WORKS

In this paper, we propose a hybrid feature-based hallucination detection method that leverages both internal states and output probabilities of large language models (LLMs). This approach integrates features such as hidden layer activations, attention mechanisms, and entropy-based probability analysis to identify hallucinations with greater accuracy and robustness. By conducting experiments on the self-created EpicQA dataset, our method demonstrated superior performance compared to traditional single-feature-based methods, proving its effectiveness in detecting hallucinations in long texts. Additionally, extensive evaluations using multiple metrics confirmed that our method outperforms existing baselines, such as Predictive Probability and Entropy-based models. However, our current analysis is confined to Transformer-based architectures. Future work will focus on extending the analysis to other model types and addressing challenges in black-box models like GPT-4, for which internal states are inaccessible. This advancement is crucial for improving the reliability, safety, and trustworthiness of LLM-generated content in diverse applications

ACKNOWLEDGEMENT

This work is supported by the Youth Innovation Promotion Association CAS (No.2023166) and the Informatization Plan of Chinese Academy of Sciences (Grant No. CAS-WX2021SF-0508).

Authorized licensed use limited to: M S RAMAIAH INSTITUTE OF TECHNOLOGY. Downloaded on October 01,2026 at 09:59:34 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko *et al.*, "Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," *arXiv preprint arXiv:2401.08671*, 2024.

[2] J.-Y. Yao, K.-P. Ning, Z.-H. Liu, M.-N. Ning, and L. Yuan, "Llm lies: Hallucinations are not bugs, but features as adversarial examples," *arXiv preprint arXiv:2310.01469*, 2023.

[3] D. P. Panagoulias, M. Virvou, and G. A. Tsihrintzis, "Evaluating llm–generated multimodal diagnosis from medical images and symptom analysis," *arXiv preprint arXiv:2402.01730*, 2024.

[4] I. Cheong, K. Xia, K. K. Feng, Q. Z. Chen, and A. X. Zhang, "(a) i am not a lawyer, but...: Engaging legal experts towards responsible llm policies for legal advice," in *The 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024, pp. 2454–2469.

[5] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6491–6501.

[6] A. Azaria and T. Mitchell, "The internal state of an LLM knows when it's lying," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 967–976. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.68

[7] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, 2024.

[8] E. Quevedo, J. Yero, R. Koerner, P. Rivas, and T. Cerny, "Detecting hallucinations in large language model generation: A token probability approach," *arXiv preprint arXiv:2405.19648*, 2024.

[9] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson *et al.*, "Language models (mostly) know what they know," *arXiv preprint arXiv:2207.05221*, 2022.

[10] S. Tonmoy, S. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.

[11] T. Guan, F. Liu, X. Wu, R. Xian, Z. Li, X. Liu, X. Wang, L. Chen, F. Huang, Y. Yacoob *et al.*, "Hallusionbench: an advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 14 375–14 385.

[12] W. Su, C. Wang, Q. Ai, Y. Hu, Z. Wu, Y. Zhou, and Y. Liu, "Unsupervised real-time hallucination detection based on the internal states of large language models," in *Findings of the Association for Computational Linguistics ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand and virtual meeting: Association for Computational Linguistics, Aug. 2024, pp. 14 379–14 391. [Online]. Available: https://aclanthology.org/2024.findings-acl.854

[13] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[14] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9004–9017. [Online]. Available: https://aclanthology.org/2023.emnlp-main.557

[15] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, E. Goffinet, D. Heslow, J. Launay, Q. Malartic *et al.*, "Falcon-40b: an open large language model with state-of-the-art performance. 2023," *URL https://falconllm. tii. ae*, 2023.

[16] B. Wang and A. Komatsuzaki, "Gpt-j-6b: A 6 billion parameter autoregressive language model," 2021.

[17] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[18] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[19] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[20] P. Manakul, A. Liusie, and M. J. Gales, "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models," *arXiv preprint arXiv:2303.08896*, 2023.

Authorized licensed use limited to: M S RAMAIAH INSTITUTE OF TECHNOLOGY. Downloaded on October 01,2026 at 09:59:34 UTC from IEEE Xplore. Restrictions apply.